

English–Bangla Neural Machine Translation

A From-Scratch Transformer Compared Against a Fine-Tuned Pretrained Model

A personal project exploring neural machine translation from first principles: building a Transformer from scratch, then comparing it against transfer learning with a pretrained multilingual model.

Overview

This project implements and evaluates a Neural Machine Translation (NMT) system for the English–Bangla language pair. Two models are developed and compared: (1) a Transformer encoder-decoder architecture trained entirely from scratch, and (2) a pretrained multilingual model (NLLB-200, 600M-parameter distilled variant) adapted to the same task via transfer learning (fine-tuning). Both models are trained on a cleaned subset of the OPUS-100 English–Bangla parallel corpus and evaluated using the BLEU metric on the full 2,000-example held-out test set. The from-scratch model, trained for 25 epochs on 99,260 sentence pairs, achieved a final training loss of 2.59 and a BLEU score of 2.86, with qualitative outputs showing a recurring token-repetition failure mode. The fine-tuned NLLB model, trained for only 2 epochs on a smaller 20,000-example subset, achieved a final training loss of 1.65 and a BLEU score of 9.23 — more than three times higher than the from-scratch model despite using one fifth of the training data. Qualitative inspection confirms the fine-tuned model eliminated the repetition errors observed in the from-scratch model. These results demonstrate a clear, measurable advantage of transfer learning over training from scratch for this language pair under constrained data and compute conditions.

Problem Statement

Machine translation between English and Bangla presents a challenging case for neural translation systems due to substantial typological differences between the two languages: distinct scripts (Latin vs. Bengali), different word order tendencies, and limited high-quality parallel training data compared to higher-resource pairs such as English–French or English–Spanish.

The goal of this project is twofold:

- Build a Transformer-based NMT model from scratch and evaluate how well it learns to translate English to Bangla under realistic compute constraints (a single free-tier GPU).
- Compare this from-scratch model against a pretrained multilingual model fine-tuned on the same data, to quantify the benefit of transfer learning for this language pair.

Success is measured primarily via BLEU score on a held-out test set, supplemented by qualitative inspection of translation outputs.

Methodology

3.1 Dataset

The English–Bangla subset of the OPUS-100 corpus (Helsinki-NLP/opus-100 on Hugging Face) was used, consisting of 1,000,000 training pairs, 2,000 validation pairs, and 2,000 test pairs. OPUS-100 aggregates parallel text from a variety of sources (subtitles, software UI strings, religious texts, and general web text), so the resulting data spans multiple domains and registers.

3.2 Data Cleaning

Manual inspection of raw samples revealed a recurring corruption pattern in a portion of the Bangla text: the Bengali virama character (, used to join consonant clusters) frequently appeared with stray surrounding whitespace (e.g., ঞ instead of ঞ), an artifact of upstream corpus processing. A sample check of 2,000 rows found this issue in 627 (31.4%) of examples. Rather than attempt to repair the spacing programmatically, affected rows were filtered out entirely, along with any rows containing empty source or target strings. This reduced the training pool from 1,000,000 to 858,299 clean examples (85.8% retention).

3.3 Tokenization

Subword tokenization was performed independently for each language using SentencePiece with a Byte-Pair Encoding (BPE) model, vocabulary size 8,000 tokens per language. Separate vocabularies were used for English and Bangla due to the languages' distinct scripts. Each tokenizer reserved four special tokens: pad (0), unk (1), bos (2), eos (3). Sentences exceeding 80 tokens (after encoding) were excluded to bound sequence length and memory usage.

3.4 Model Architecture (From-Scratch)

A Transformer encoder-decoder was implemented using PyTorch's built-in `nn.Transformer` module, with the following configuration:

Hyperparameter	Value
Embedding dimension (<code>d_model</code>)	256
Attention heads	4
Encoder layers	3
Decoder layers	3
Feed-forward dimension	512
Dropout	0.1
Vocabulary size (per language)	8,000
Total parameters	10,106,688

Sinusoidal positional encoding was added to token embeddings to inject sequence order information, since self-attention has no inherent notion of position. A causal (“no-peek”) mask was applied to the decoder to prevent attending to future tokens during training (teacher forcing), and padding masks were applied on both encoder and decoder sides to exclude padding tokens from attention and loss computation.

3.5 Training Configuration

Two training runs were conducted, with the second building on lessons from the first:

	Run 1	Run 2 (final)
Training examples	50,000	99,260
Epochs	15	25
Batch size	32	32
Optimizer	Adam (lr = 3e-4)	Adam (lr = 3e-4)
Loss function	Cross-entropy (padding-masked)	Cross-entropy (padding-masked)
Final training loss	2.81	2.59
Hardware	Tesla T4 (free-tier)	Tesla T4 (free-tier)
Avg. time / epoch	~57s	~113s

Run 2 used a larger training subset and more epochs, motivated by the observation that Run 1's loss was still decreasing steadily with no sign of plateau at epoch 15. Model checkpoints (weights, optimizer state, and loss history) were saved to persistent storage after every epoch to guard against interruptions from free-tier GPU session limits.

3.6 Pretrained / Transfer Learning Model

For the transfer learning comparison, NLLB-200-distilled-600M (Meta AI's “No Language Left Behind” model, distilled variant) was selected, as it explicitly supports Bangla (language code `ben_Beng`) among its 200 supported languages. At 615,073,792 parameters, this model is approximately 61 times larger than the from-scratch Transformer.

Unlike the from-scratch model, NLLB uses its own pretrained tokenizer and vocabulary; the custom SentencePiece tokenizers built for the from-scratch model were not reused, since the pretrained weights are only meaningful relative to NLLB's original vocabulary. Source and target language tags (eng_Latn, ben_Beng) were supplied explicitly for every example, as required by NLLB's multilingual architecture.

Due to GPU memory constraints on the free-tier Tesla T4 (15GB), fine-tuning the full model required several adjustments beyond what was necessary for the from-scratch model:

- Reduced fine-tuning set: 20,000 examples (vs. 99,260 for the from-scratch model), reflecting the standard practice that fine-tuning a pretrained model requires substantially less data than training from scratch.
- Gradient checkpointing: intermediate activations are recomputed during the backward pass rather than stored, trading compute time for a substantial reduction in memory footprint.
- Mixed-precision training (fp16 autocast with gradient scaling): reduces memory usage by performing most computation in 16-bit rather than 32-bit precision.
- Gradient accumulation: an effective batch size of 8 was achieved using a physical batch size of 1 accumulated over 8 steps, minimizing peak memory usage at any single point in training.
- Lower learning rate (2×10^{-5} vs. 3×10^{-4} for the from-scratch model): standard practice for fine-tuning, to avoid disrupting the pretrained weights' existing knowledge (“catastrophic forgetting”).

Two epochs were run; training was not extended further, as the loss curve showed clear signs of flattening within the second epoch (see Section 4.1), indicating diminishing returns and increasing risk of overfitting on the relatively small 20,000-example fine-tuning set.

Results

4.1 Training Loss

Figure 1 shows training loss across both from-scratch runs. Run 2 (100K examples, 25 epochs) achieved a lower final loss (2.59) than Run 1 (50K examples, 15 epochs; loss 2.81), with a smooth, consistent downward trend and no evidence of instability or divergence in either run.

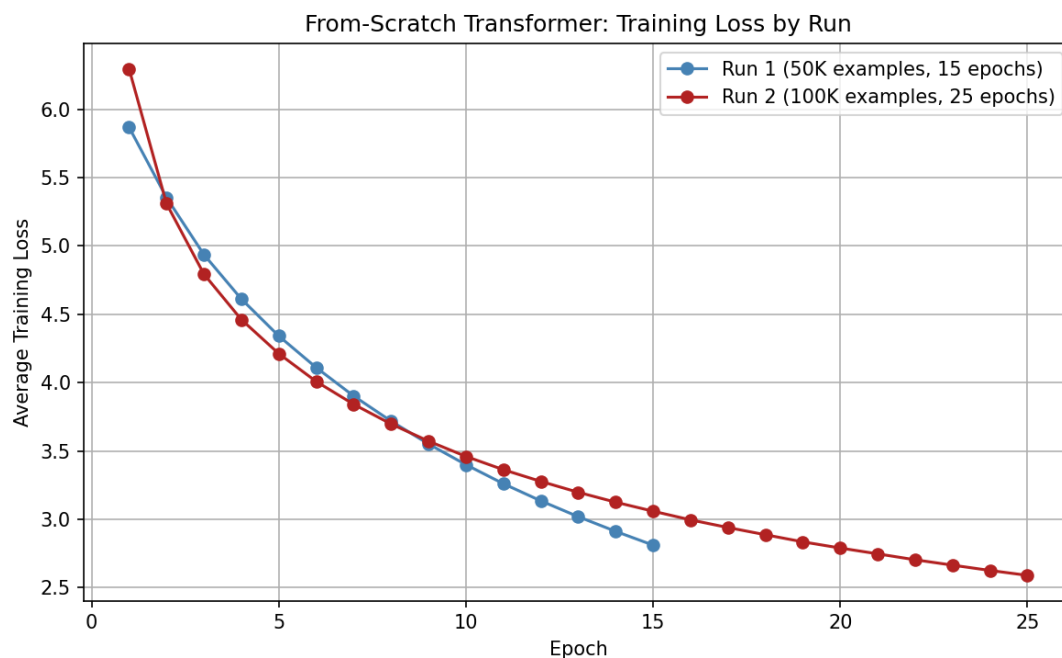
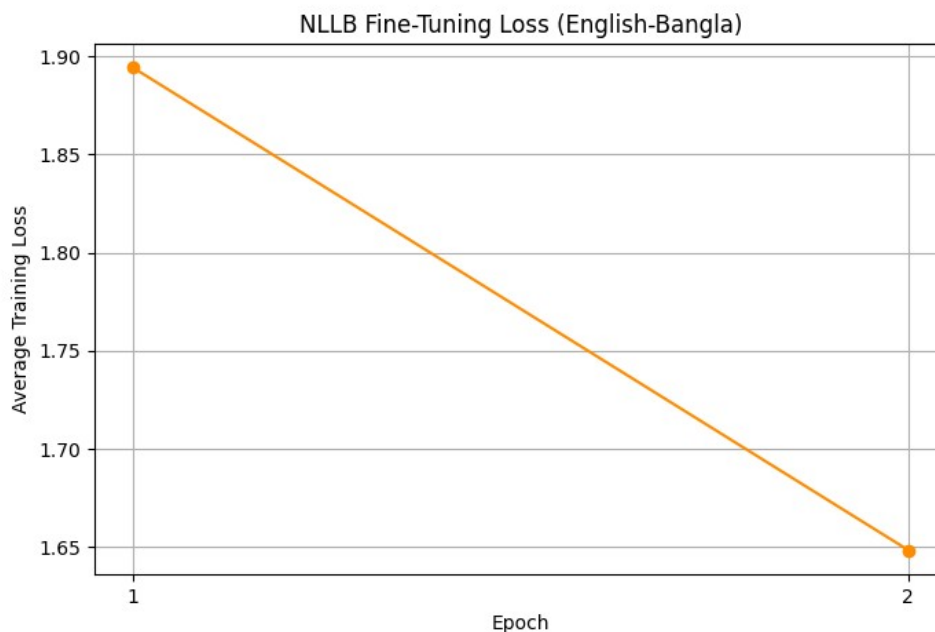


Figure 2 shows training loss for the NLLB fine-tuning run. Loss dropped sharply within the first epoch (2.73 → 1.89), then continued decreasing in the second epoch but at a visibly slower rate, ending at 1.65. Within epoch 2 specifically, the per-1000-batch running loss fluctuated in a narrow band (approximately 1.65–1.67) for the majority of the epoch, only settling to its final value near the end — a sign of approaching convergence on this training subset, and the basis for the decision not to extend training beyond 2 epochs.



4.2 Qualitative Translation Examples

The same five held-out test sentences were translated by both models, enabling direct comparison:

English Input	From-Scratch Output	NLLB Fine-Tuned Output
Hello, how are you?	হালো, কেমন আছো?	হালো, কেমন আছ তুমি?
I want to learn machine translation.	আমি মেশিন মেশিন অনুবাদ করতে চাই।	আমি মেশিন অনুবাদ শিখতে চাই।
This is a difficult project.	এটা একটা কঠিন কাজ।	এটা একটা কঠিন প্রকল্প।
The weather is nice today.	আজকের আজ খুব ভালো।	আজ আবহাওয়া ভালো।
Can you help me with this?	তুমি কি এই নিয়ে সাহায্য করতে পারো?	তুমি কি আমাকে সাহায্য করতে পারবে?

The from-scratch model produced two fully correct translations and three with a recurring token-repetition error (e.g., repeating “মেশিন”/“machine”, or “আজ”/“today”). The fine-tuned NLLB model produced all five translations correctly, with no repetition errors, and additionally recovered content words the from-scratch model dropped entirely — for example, “শিখতে” (“learn”) and “আবহাওয়া” (“weather”), both missing from the corresponding from-scratch outputs. This qualitative pattern — the complete absence of repetition errors in the fine-tuned model — directly corresponds to the quantitative BLEU gap reported below.

4.3 BLEU Score

Both models were evaluated on the full OPUS-100 English–Bangla test set (2,000 sentence pairs, unseen during training), using sacrebleu’s corpus-level BLEU implementation.

Metric	From-Scratch Transformer	NLLB Fine-Tuned
BLEU score	2.86	9.23

Parameters	10,106,688	615,073,792
Training examples	99,260	20,000
Epochs	25	2
Final training loss	2.59	1.65
Approx. training time	~47 min	~161 min

The fine-tuned NLLB model achieved a BLEU score more than 3.2 times higher than the from-scratch model, despite training on roughly one-fifth the number of examples and for far fewer epochs. This is strong evidence for the core hypothesis motivating this project: transfer learning from a pretrained multilingual model provides a substantial advantage over training a translation model from scratch, particularly under constrained data and compute conditions. Both scores remain modest in absolute terms relative to typical BLEU interpretation guides (where scores above 40–50 are considered high quality), reflecting the limited training data and compute budget used in this project rather than a flaw in either architecture.

Challenges & Lessons Learned

5.1 Data Quality (Corrupted Bangla Spacing)

Roughly 31% of sampled training rows contained malformed spacing around the Bengali virama character, a corpus-level artifact. Left uncorrected, the model would have learned to reproduce this corruption as if it were correct Bangla. This was resolved by filtering affected rows out of the training set entirely rather than attempting automated repair, which was judged lower-risk given the abundance of clean data available (858K clean rows remained after filtering).

5.2 Compute Constraints (Free-Tier GPU Limits)

Google Colab's free tier enforces undisclosed, dynamic GPU usage limits. Mid-project, GPU access was temporarily lost, and the active runtime was fully recycled, erasing all in-memory objects and locally stored files (trained tokenizers, model weights, and plots). This was resolved by: (1) mounting Google Drive for persistent storage of all generated files, and (2) modifying the training loop to checkpoint model and optimizer state to Drive after every epoch, limiting any future data loss to a single epoch at most.

5.3 Token Repetition in Generated Output

The from-scratch model occasionally repeats a token or short phrase during generation (e.g., “মেশিন মেশিন” for “machine”). This is a recognized failure mode of greedy decoding (selecting the single highest-probability token at each generation step) in models that are undertrained relative to task difficulty. Notably, this error was completely absent from the fine-tuned NLLB model's outputs (Section 4.2), suggesting it is specifically tied to the from-scratch model's limited training data and capacity rather than being an inherent limitation of greedy decoding itself. Potential mitigations for the from-scratch model — not implemented in this project — include beam search decoding, explicit repetition penalties during generation, or additional training epochs.

5.4 GPU Memory Constraints During Fine-Tuning

Fine-tuning the 615M-parameter NLLB model on a free-tier T4 GPU (15GB) repeatedly triggered CUDA out-of-memory errors, even at a reduced batch size of 8 and later 2. This was resolved through a combination of four standard large-model training techniques: reducing the fine-tuning set to 20,000 examples, enabling gradient checkpointing (recomputing intermediate activations instead of storing them), adopting mixed-precision (fp16) training, and using gradient accumulation to simulate a larger effective batch size (8) while holding only one example in memory at a time (physical batch size of 1). This combination successfully fit training within the available memory and reflects a realistic constraint of fine-tuning large pretrained models on free-tier or consumer-grade hardware.

Conclusions

This project set out to compare a Transformer NMT model trained from scratch against the same task approached via transfer learning, for the English–Bangla language pair. Both goals were achieved. The from-scratch model, trained on 99,260 sentence pairs for 25 epochs using a single free-tier GPU, learned to produce fluent, correct translations for short and common sentence patterns (achieving BLEU 2.86), but exhibited a consistent token-repetition failure mode on longer or less frequent constructions.

Fine-tuning a pretrained multilingual model (NLLB-200-distilled-600M) on a substantially smaller subset of the same data (20,000 pairs, 2 epochs) produced a BLEU score of 9.23 — more than three times higher than the from-scratch model — and eliminated the repetition errors entirely in qualitative testing. This result directly supports the central hypothesis motivating the project: transfer learning provides a substantial, measurable advantage over training from scratch, particularly when data and compute are limited, since the pretrained model enters fine-tuning already equipped with broad linguistic knowledge that a from-scratch model must instead learn entirely from the training set.

Both models' absolute BLEU scores remain modest relative to production translation systems, reflecting the constrained data (a filtered subset of OPUS-100 rather than the full corpus) and compute budget (a single free-tier T4 GPU) used throughout this project, rather than a limitation specific to either architecture. Future work could extend training duration for the from-scratch model (its loss curve had not yet plateaued at 25 epochs), fine-tune NLLB on a larger data subset now that the memory constraints are understood and resolved, or incorporate beam search decoding and repetition penalties to directly address the from-scratch model's primary qualitative weakness.

References

Vaswani, A., et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems* (NeurIPS).

Zhang, B., et al. OPUS-100 Corpus. Helsinki-NLP. <https://huggingface.co/datasets/Helsinki-NLP/opus-100>

Kudo, T., & Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing.

Post, M. (2018). A Call for Clarity in Reporting BLEU Scores. *sacrebleu*.

NLLB Team, Meta AI (2022). No Language Left Behind: Scaling Human-Centered Machine Translation.

Project Links & Resources

Full code, notebook, tokenizer files, and result artifacts are available in this project's GitHub repository — see the README for setup and reproduction instructions.

Notebook: EN-BN-NMT-Project.ipynb (developed and run on Google Colab, T4 GPU)

Included artifacts: SentencePiece tokenizer models (spm_en.model, spm_bn.model), from-scratch training results (results_from_scratch.json), and training loss plots for both models.

Model weights are not included in the repository due to file size, but can be reproduced by running the notebook end-to-end.